

İşletim Sistemleri

Bölüm 1: Giriş (Introduction) Bölüm Sonu Soruları — Tam Çözümler & Açıklamalar

Kaynak	Konu	Dil	Seviye
Modern Operating Systems (Tanenbaum)	Bölüm 1 — Giriş	Türkçe Açıklama	Lisans



Bu belge, işletim sistemleri dersinin 1. Bölümüne ait 38 adet bölüm sonu sorusunun Türkçe tam çözümlerini içermektedir. Teknik terimler İngilizce olarak korunmuştur. Kavramsal açıklamalar, karşılaştırma tabloları, kod örnekleri ve quiz soruları mevcuttur.

İçindekiler

- 1-2. İşletim Sisteminin Temel İşlevleri ve Çoklu Programlama
- 3. İşletim Sistemi Türleri ve Uygulama Alanları
- 4. Cache Bellek — Satır Önbellekleme
- 5. Spooling Nedir?
- 6. DMA Olmadan Çoklu Programlama
- 7. İkinci Nesil Bilgisayarlarda Zaman Paylaşımı
- 8. Ayrıcalıklı G/Ç Komutları
- 9. Video RAM Hesaplamaları
- 10. Çelişen Tasarım Hedefleri
- 11. Kernel ve Kullanıcı Modu
- 12. Disk Geometrisi ve Erişim Süresi Hesabı
- 13. Çok CPU / Hyper-threading — Program Tamamlanma Süresi
- 14. Kişisel Bilgisayar ile Anabilgisayar OS Farkları
- 15. Pipeline Performansı
- 16. Ortalama Bellek Erişim Süresi
- 17. Disk Yazma — Süreç Bloke Edilmeli mi?
- 18. Trap ile Interrupt Farkı
- 19. Dolmayan Dizine Dosya Sistemi Bağlama
- 20. Sistem Çağrısının Amacı
- 21-22. Dosya Sistemi Bağlama Tasarımı ve Sistem Çağrısı Hataları
- 23. Kaynak Çoğullama Türleri
- 24. write() Sistem Çağrısı — Kısmi Yazım
- 25. lseek + read Örneği
- 26. Dosya Okuma Süresi Hesabı
- 27. Blok ve Karakter Özel Dosyaları
- 28. Kütüphane Prosedürü ile Sistem Çağrısı Adlandırması
- 29. Client-Server Modeli — Tek Bilgisayar
- 30. Sistem Çağrısı Farkındalığı
- 31. UNIX → Win32 Dönüşüm Sonuçları
- 32. Taşınabilir İşletim Sistemi Tasarımı
- 33. Politika ve Mekanizma Ayrımı — Microkernel
- 34. Sanal Makinelerin Dezavantajları
- 35. Birim Dönüşümleri
- 36-38. Programlama Egzersizleri (Shell, Fork Bomb, od)
- ♦ Açık Uçlu Quiz Soruları ve Cevapları

Soru 1-2 — İşletim Sisteminin Temel İşlevleri & Çoklu Programlama

[EN] What are the two main functions of an operating system?

[TR] İşletim sisteminin iki temel işlevi nedir?

◆ **Tanım (Definition):** İşletim sistemi (OS), donanım kaynaklarını yöneten ve uygulama programlarına hizmet sunan sistem yazılımıdır.

İşlev	Açıklama	Örnek
1. Kaynak Yöneticisi (Resource Manager)	CPU, bellek, disk, G/Ç aygıtlarını adil ve verimli biçimde programlar arasında paylaştırır.	CPU scheduler, memory allocator
2. Genişletilmiş Makine (Extended Machine)	Donanımın karmaşık ayrıntılarını gizleyerek programcıya soyut, kullanımı kolay bir arayüz sunar.	Dosya sistemi, POSIX API

[EN] What is multiprogramming?

[TR] Çoklu programlama (multiprogramming) nedir?

Çoklu programlama, birden fazla programın bellekte *aynı anda* bulunduğu ve CPU'nun boş kaldığı anlarda (örn. G/Ç bekleme) başka bir programa geçiş yaparak verimliliği artıran tekniktir.

✓ **Örnek:** Program A diskten veri okurken (10 ms bekliyor) CPU, Program B'yi çalıştırabilir. Bu sayede CPU kullanım oranı %30'dan %90'a çıkabilir.

Kavram	Multiprogramming	Multitasking (Zaman Paylaşımı)
Geçiş Tetikleyicisi	G/Ç bekleme (gönüllü)	Timer interrupt (zorunlu)
Amaç	CPU verimliliği	Kullanıcı etkileşimi
Yanıt Süresi	Önemli değil	Hızlı (ms mertebesinde)

Soru 3 — İşletim Sistemi Türleri ve Uygulama Alanları

[EN] In Sec. 1.4, nine different types of operating systems are described. Give a list of possible applications for each of these systems (at least one for each).

[TR] 9 farklı işletim sistemi türü için olası uygulama alanlarını listeleyin.

OS Türü	Açıklama	Uygulama Örneği
Mainframe OS	Toplu iş ve yüksek hacimli veri işleme	Banka işlem merkezi, havayolu rezervasyon
Server OS	Ağ hizmetleri, dosya/baskı paylaşımı	Linux sunucu, Windows Server, web hosting

Multiprocessor OS	Çok işlemcili sistemler	Bilimsel simülasyon, veritabanı sunucuları
Personal Computer OS	Tek kullanıcı, GUI odaklı	Windows 11, macOS, Ubuntu Desktop
Real-Time OS (RTOS)	Kesin zaman kısıtları	Otomotiv ABS, endüstriyel PLC, uçak aviyonığı
Embedded OS	Kaynak kısıtlı donanım	Akıllı saatler, buzdolabı kontrol kartı
Smart Card OS	Çip üzeri minimum bellek	SIM kart, kredi kartı, e-pasaport
Sensor Node OS (TinyOS vb.)	Çok düşük güç tüketimi	Akıllı tarım sensörleri, IoT ağları
Handheld / Mobile OS	Pil ömrü, dokunmatik arayüz	Android, iOS — akıllı telefon, tablet

Soru 4 — Cache Bellek — Satır (Cache Line) Önbellekleme

[EN] To use cache memory, main memory is divided into cache lines, typically 32 or 64 bytes long. An entire cache line is cached at once. What is the advantage of caching an entire line instead of a single byte or word at a time?

[TR] Tek bayt yerine tüm cache satırının önbelleğe alınmasının avantajı nedir?

Temel avantaj **uzamsal yerlik (spatial locality)** ilkesinden faydalanmaktır. Programlar genellikle belleği *ardışık* biçimde erişir (dizi döngüleri, kod akışı vb.). Bir satırı tümüyle çekmek:

Yaklaşım	Avantaj	Dezavantaj
Tek bayt önbellek	Minimum transfer	Çok sayıda miss → yüksek gecikme
Tam cache line (32-64 B)	Komşu veriler hazır; az miss	Kullanılmayacak veri de transfer edilir

★ **Örnek:** `int arr[1000]` üzerinde döngü: `arr[0]` erişildiğinde 64-byte cache line alınır; `arr[1..15]` de cache'e gelir. Sonraki 15 erişim cache hit olur, RAM'e gidilmez.

Soru 5 — Spooling Nedir?

[EN] What is spooling? Do you think that advanced personal computers will have spooling as a standard feature in the future?

[TR] Spooling nedir? Gelecekte kişisel bilgisayarlarda standart özellik olacak mı?

Spooling (Simultaneous Peripheral Operations On-Line): Yavaş çalışan çıkış aygıtları (özellikle yazıcı) için verilerin önce diske yazılıp ardından arka planda aygıtı iletildiği tampon mekanizmasıdır.



Tarihsel bağlam: İlk bilgisayarlarda yazıcıya doğrudan yazan program, yazıcı çalışana kadar beklemek zorundaydı. Spooling bu beklemeyi ortadan kaldırdı.

Gelecek için değerlendirme: Modern kişisel bilgisayarlarda spooling halihazırda standart bir özelliktir (Windows Print Spooler, CUPS). Yazıcı, faks ve ses arabelleği gibi yavaş aygıtlar var olduğu sürece spooling gerekli olmaya devam edecektir.

Soru 6 — DMA Olmadan Çoklu Programlama

[EN] On early computers, every byte of data read or written was handled by the CPU (i.e., there was no DMA). What implications does this have for multiprogramming?

[TR] DMA olmadan çoklu programlama nasıl etkilenir?

DMA (Direct Memory Access) olmadan CPU her bayt transferini kendisi yapar. Bu durum şu sonuçları doğurur:



Sorun: CPU, G/Ç transferi sırasında tamamen meşgul olur. Başka programa geçiş yapılamaz çünkü CPU'nun dikkatini veriler arasında bölmek mümkün değildir. Bu nedenle çoklu programlamanın faydası büyük ölçüde kaybolur — CPU boşta beklemek yerine bayt taşımayla meşguldür.

DMA ile CPU yalnızca transferi başlatır ve tamamlandığında interrupt alır; aradaki sürede başka işlem çalıştırabilir → gerçek paralel G/Ç + hesaplama mümkün olur.

Soru 7 — İkinci Nesil Bilgisayarlarda Zaman Paylaşımı

[EN] Why was timesharing not widespread on second-generation computers?

[TR] Zaman paylaşımı (timesharing) ikinci nesil bilgisayarlarda neden yaygınlaşmadı?

Engel	Açıklama
Donanım maliyeti	1960'larda bellek çok pahalıydı; birden fazla programı bellekte tutmak ekonomik değildi.
İşlemci hızı	CPU'lar o dönemde yavaştı; kullanıcı gecikmesi kabul edilemez seviyedeydi.
Güvenlik mekanizmaları eksik	Kullanıcıları birbirinden koruyacak donanım desteği (memory protection, mode bits) gelişmemişti.
Toplu iş (batch) kültürü	Operatörler ve kullanıcılar toplu iş sistemlerine alışkındı; interaktif çalışma alışkanlığı yoktu.

Soru 8 — Ayrıcalıklı G/Ç Komutları (Privileged I/O Instructions)

[EN] Instructions related to accessing I/O devices are typically privileged instructions, that is, they can be executed in kernel mode but not in user mode. Give a reason why these instructions are privileged.

[TR] G/Ç komutları neden ayrıcalıklı (privileged) olmalıdır?

G/Ç komutları kernel modda çalışmalıdır çünkü:



Güvenlik: Kullanıcı programlarının doğrudan disk, ağ kartı veya klavye gibi aygıtlara erişmesine izin verilseydi, başka kullanıcıların verilerine müdahale edebilirlerdi. OS, tüm G/Ç isteklerini denetleyerek erişim kontrolü uygular (hangi süreç hangi dosyaya yazabilir gibi).



Kararlılık (Stability): Hatalı bir kullanıcı programı yanlış G/Ç komutu göndererek donanımı kilitleyebilir veya sistemi çökertebilir. Kernel bu riski doğrulama yaparak önler.

Soru 9 — Video RAM Hesaplamaları

[EN] How much video RAM is needed to support a 25-line $\times$ 80-row character monochrome text screen? How much for a 1024 $\times$ 768-pixel 24-bit color bitmap? What was the cost at 1980 prices (\$5/KB)? How much is it now?

[TR] Metin ekranı ve bitmap ekran için gereken video RAM miktarlarını ve maliyetlerini hesaplayın.

Hesaplamalar:

```
# 1. Metin Ekranı (25 satır  $\times$  80 sütun, monochrome)
Her karakter = 1 bayt (ASCII kodu)
Toplam = 25  $\times$  80 = 2.000 bayt  $\approx$  2 KB

# 2. Bitmap Ekranı (1024  $\times$  768, 24-bit renk)
Her piksel = 3 bayt (RGB)
Toplam = 1024  $\times$  768  $\times$  3 = 2.359.296 bayt  $\approx$  2,25 MB

# 3. 1980 Maliyeti ($5 / KB)
Metin: 2 KB  $\times$  $5 = $10
Bitmap: 2.250 KB  $\times$  $5 = $11.250 (çok pahalı!)

# 4. Günümüz Maliyeti (~$3-5 / GB)
Bitmap: 2,25 MB / 1024  $\times$  $4/GB  $\approx$  $0,000009  $\rightarrow$  ihmal edilebilir
```



1980'lerde 1024 $\times$ 768 bitmap ekran için gereken RAM maliyeti ($\approx$ \$11.250), o dönem maaşlarının üstünde olduğundan GUI'ler popülerleşemedi. Günümüzde aynı RAM miktarı kuruluşlarla satın alınabilmektedir.

Soru 10 — Çelişen Tasarım Hedefleri

[EN] Give an example of two design goals that may contradict one another.

[TR] Birbiriyle çelişebilecek iki tasarım hedefi örneği verin.

Hedef Çifti	Neden Çelişir?
Güvenlik ↔ Kullanım Kolaylığı	Güvenlik arttıkça şifre, onay vb. adımlar artar → kullanıcı deneyimi zorlaşır. (Örn: iki faktörlü kimlik doğrulama)
Kaynak Verimliliği ↔ Yanıt Hızı	CPU'yu tam doldurmak verimli ama kullanıcı isteklerine anında yanıt vermeyi güçleştirir. (Batch vs Interactive)
Taşınabilirlik ↔ Performans	Platforma özgü optimizasyonlar taşınabilirliği azaltır; taşınabilir kod daha yavaş çalışabilir.

Soru 11 — Kernel Modu ve Kullanıcı Modu

[EN] What is the difference between kernel and user mode? Explain how having two distinct modes aids in designing an operating system.

[TR] Kernel modu ile kullanıcı modu arasındaki fark nedir ve iki modun varlığı OS tasarımına nasıl katkı sağlar?

Özellik	Kernel Modu	Kullanıcı Modu
Erişim Yetkisi	Tüm donanım ve bellek	Yalnızca kendi adres alanı
Ayrıcalıklı Komutlar	Çalıştırabilir (I/O, MMU)	Çalıştıramaz → trap
Geçiş	Interrupt / Sistem çağrısı	Normal program akışı
Çöküş Etkisi	Sistem çöker	Yalnızca ilgili süreç sonlanır
Örnek Yazılım	OS çekirdeği, sürücüler	Tarayıcı, metin editörü

✓ **Tasarım Faydası:** İki mod, hatalı kullanıcı programlarının sistemi bozmasını engeller (protection). OS'nin kritik veri yapıları (sayfa tabloları, süreç listesi) yalnızca kernel modda değiştirilebilir → güvenilir ve sağlam bir sistem.

Soru 12 — Disk Geometrisi ve Erişim Süresi Hesabı

[EN] A 255-GB disk has 65,536 cylinders with 255 sectors per track and 512 bytes per sector. How many platters and heads does this disk have? Assuming average cylinder seek time 11 msec, average rotational delay 7 msec, reading rate 100 MB/sec, calculate the average time to read 100 KB.

[TR] Disk geometrisini ve 100 KB okuma süresini hesaplayın.

Disk Geometrisi

Toplam kapasite = 255 GB = $255 \times 1024 \times 1024 \times 1024$ bayt

≈ 273.804.165.120 bayt

Silindir başına kapasite = 65.536 silindirler

Sektör başına = 512 bayt

İz başına bayt = sektör/iz × bayt/sektör = 255 × 512 = 130.560 bayt

Silindir başına bayt = iz/silindir × 130.560

Yüzey sayısı (platter yüzü) = Toplam / (65536 × 130560)

= 273.804.165.120 / 8.553.553.920

≈ 32 yüzey → 16 platter, 32 kafa (head)

100 KB Okuma Süresi

Seek süresi = 11 ms (ortalama)

Dönme gecikmesi = 7 ms (ortalama)

Transfer süresi = 100 KB / 100 MB/s = 0,1 ms

Toplam = 11 + 7 + 0,1 = 18,1 ms



Disk erişiminde en büyük zaman dilimini seek + rotational delay (mekanik gecikmeler) oluşturur; gerçek veri transferi çoğu zaman ihmal edilebilir küçüklüktedir.

Soru 13 — Çok CPU / Hyper-threading — Program Tamamlanma Süresi

[EN] Consider a system with two CPUs, each having two threads (hyperthreading). Three programs P0, P1, P2 with run times 5, 10, 20 msec. How long will it take to complete? (100% CPU-bound, no blocking, no CPU switch)

[TR] 2 CPU × 2 thread = 4 mantıksal çekirdek. P0=5ms, P1=10ms, P2=20ms tamamlanma süresi nedir?

4 mantıksal thread mevcut, 3 program var. Hepsi aynı anda başlayabilir:

Thread 0 (CPU0-T0) → P0 çalışır: 0-5 ms → tamamlandı

Thread 1 (CPU0-T1) → P1 çalışır: 0-10 ms → tamamlandı

Thread 2 (CPU1-T0) → P2 çalışır: 0-20 ms → tamamlandı

Thread 3 (CPU1-T1) → boşta

Toplam süre = max(5, 10, 20) = 20 ms



Cevap: 20 ms. En uzun program P2'dir ve CPU beklemeye gerek kalmadan çalışır. Hyper-threading burada 4. thread sağladığı için hiçbir program sıraya girmek zorunda kalmaz.

Soru 14 — Kişisel Bilgisayar OS'si vs. Mainframe OS'si

[EN] List some differences between personal computer operating systems and mainframe operating systems.

[TR] Kişisel bilgisayar ve anabilgisayar işletim sistemlerinin farklarını listeleyin.

Özellik	PC OS (Windows/Linux)	Mainframe OS (z/OS)
Kullanıcı Sayısı	Genellikle 1 kullanıcı	Yüzlerce-binlerce eşzamanlı
Güvenilirlik (RAS)	Yılda birkaç kez yeniden başlatma normal	7/24 çalışma, saniyeler içinde failover
İş Türü	İnteraktif, multimedya	Toplu iş (batch), OLTP
Kaynak Koruma	Sınırlı	Kesin yalıtım, SLA garantisi
Depolama Yönetimi	NTFS, ext4	VSAM, SMS, büyük I/O bantları
Fiyat	Onlarca - yüzlerce dolar	Milyonlarca dolar

Soru 15 — Pipeline Performansı

[EN] A computer has a pipeline with four stages. Each stage takes 1 nsec. How many instructions per second can this machine execute?

[TR] 4 aşamalı pipeline, her aşama 1 ns. Saniyede kaç komut çalışır?

Pipeline döngüsü = 1 ns (en yavaş aşama belirler)

Her döngüde 1 komut tamamlanır (steady-state)

Saniyedeki komut sayısı = $1 / (1 \times 10^{-9} \text{ s}) = 10^9 = 1 \text{ Milyar komut/s}$
= 1 GIPS (Giga Instructions Per Second)

NOT: İlk 3 döngüde pipeline dolmaktadır (latency = 4 ns).

Uzun programlarda bu başlangıç maliyeti ihmal edilir.

i

4 aşamalı pipeline teorik olarak pipeline'sız sisteme göre 4x hızlıdır. Gerçekte branch, data hazard gibi sorunlar bu oranı düşürür.

Soru 16 — Ortalama Bellek Erişim Süresi

[EN] Cache: 2 nsec, RAM: 10 nsec, Disk: 10 msec. Cache hit rate 95%, RAM hit rate (after cache miss) 99%. What is the average time to access a word?

[TR] Ortalama kelime erişim süresini hesaplayın.

Ortalama erişim süresi hesabı:

$$T_{avg} = P(\text{cache hit}) \times T_{cache}$$
$$+ P(\text{cache miss}) \times P(\text{RAM hit}) \times T_{RAM}$$
$$+ P(\text{cache miss}) \times P(\text{RAM miss}) \times T_{disk}$$

$$T_{avg} = 0,95 \times 2$$

$$+ 0,05 \times 0,99 \times 10$$

$$+ 0,05 \times 0,01 \times 10.000.000$$

$$T_{avg} = 1,90 + 0,495 + 5.000$$

$$\approx 5.002,4 \text{ ns} \approx 5 \text{ } \mu\text{s}$$

NOT: Disk isabetsizliğinin (sadece %0,05) katkısı 5000 ns ile toplam süreye tamamen hakimdir!

Ders: Disk erişimi 10 ms olduğundan, yalnızca %0,05 olasılıklı disk miss bile ortalama süreyi saniyeler mertebesinde mikrosaniyelere çeker. Sayfa hataları (page fault) sistemleri yavaşlatan en kritik faktördür.

Soru 17 — Disk Yazma — Süreç Bloke Edilmeli mi?

[EN] When a program writes to disk, need the caller be blocked awaiting completion of the disk transfer?

[TR] Diske yazarken çağıran süreç bloke edilmeli midir?

Okuma için süreç mutlaka bloke edilmelidir: veri transfer tamamlanmadan okunan veriye erişilemez.

Yazma için bloke etme zorunlu değildir:

Durum	Açıklama
Eşzamanlı yazma (bloke)	Süreç disk transfer tamamlanana kadar bekler. Güvenli ama yavaş.
Eşzamansız yazma (bloke edilmez)	OS veriyi tampona alır, süreç çalışmaya devam eder. OS arka planda yazar. Başarısız yazma süreç tarafından hemen fark edilmez.
Write-back cache	Buffer cache sayesinde yazma görünürde anlıktır. OS periyodik olarak diske flush eder.

Sonuç: Çoğu modern OS yazma için süreç bloke etmez; ancak güç kesintisi gibi durumlarda tampondaki veriler kaybolabilir. Güvenilirlik için fsync() / msync() çağrıları kullanılır.

Soru 18 — Trap ile Interrupt Farkı

[EN] What is the key difference between a trap and an interrupt?

[TR] Trap ile interrupt arasındaki temel fark nedir?

Özellik	Trap (Yazılım Kesmesi)	Interrupt (Donanım Kesmesi)
Kaynak	Mevcut çalışan program (yazılım)	Harici donanım aygıtı
Senkronizasyon	Senkron (deterministic)	Asenkron (öngörülemez)

Tetikleyici	Sistem çağrısı, sıfıra bölme, hata	Disk tamamlandı, tuşa basıldı, NIC paketi
Amaç	OS'e kontrol geçirmek (kasıtlı)	CPU'yu acil durumdan haberdar etmek
Örnek	int 0x80 (Linux syscall)	IRQ14 (disk), IRQ1 (klavye)

Soru 19 — Dolmayan Dizine Dosya Sistemi Bağlama

[EN] Is there any reason why you might want to mount a file system on a nonempty directory?

[TR] Neden dolu bir dizine dosya sistemi bağlamak isteyebilirsiniz?

Teknik olarak bu mümkündür (Linux buna izin verir). Bağlama noktası olarak seçilen dizindeki eski içerik gizlenir, mount edilen yeni dosya sistemi ön plana geçer.

i

Olası neden: Varsayılan içerik stratejisi — dolu dizinde "yedek" ya da "varsayılan" dosyalar bulunabilir. Harici aygıt bağlanınca yeni içerik görünür; aygıt çıkarılınca eski içerik otomatik olarak döner. Ancak bu kafa karıştırıcı olduğundan pratikte genellikle boş dizin kullanılır.

Soru 20 — Sistem Çağrısının (System Call) Amacı

[EN] What is the purpose of a system call in an operating system?

[TR] İşletim sisteminde sistem çağrısının amacı nedir?

Sistem çağrısı, kullanıcı modundaki programların kernel modundaki OS hizmetlerine güvenli biçimde erişmesini sağlayan mekanizmadır. Program doğrudan donanıma erişemez; bunun yerine OS'ye istek gönderir.

Uygulama → write(fd, buf, n) → [trap] → Kernel → Disk Sürücüsü → Donanım

Kullanıcı modu: write() kütüphane fonksiyonu çağrılır

Kernel modu: OS parametreleri doğrular, G/Ç başlatır, sonucu döner

Soru 21-22 — Dosya Sistemi Bağlama Tasarımı & Sistem Çağrısı Hataları

[EN] Give one reason why mounting file systems is a better design option than prefixing path names with a drive name or number.

[TR] Dosya sistemi bağlamanın sürücü adı öneki yaklaşımına göre avantajı nedir?

Taşınabilirlik ve şeffaflık: Mount noktaları sayesinde uygulamalar hangi fiziksel aygıtta çalıştığından habersiz olur. C:\users\... gibi yol adları sürücü değiştiğinde kod değiştirmeyi gerektirir; /home/user/ ise her zaman aynı kalır.

[EN] For each of the following system calls, give a condition that causes it to fail: open, close, lseek.

[TR] open, close ve lseek sistem çağrılarının başarısız olmasına yol açan durumlar:

Sistem Çağrısı	Başarısız Olma Koşulu
open()	Dosya mevcut değil, izin yok, dosya zaten kilitli, çok fazla açık dosya (EMFILE)
close()	Geçersiz dosya tanımlayıcısı (fd), fd zaten kapalı (EBADF)
lseek()	Dosya tanımlayıcısı pipe veya socket gibi seek desteklemeyen aygıtta ait (ESPIPE)

Soru 23 — Kaynak Çoğullama Türleri

[EN] What type of multiplexing (time, space, or both) can be used for sharing the following resources: CPU, memory, SSD/disk, network card, printer, keyboard, and display?

[TR] Her kaynak için kullanılan çoğullama türünü belirtin.

Kaynak	Çoğullama Türü	Açıklama
CPU	Zaman (Time)	Her süreç belirli zaman dilimleri alır (time slicing)
Bellek (Memory)	Uzay (Space)	Her süreç kendi bellek bölgesini kullanır
SSD/Disk	Zaman + Uzay	Bloklar uzayda ayrılır; erişimler sıralanır (zaman)
Ağ Kartı (NIC)	Zaman	Paketler sırayla gönderilir/alınır
Yazıcı (Printer)	Zaman (Spooling)	İşler sırayla yazdırılır
Klavye (Keyboard)	Zaman	Giriş bir anda yalnızca bir uygulamaya yönelir
Ekran (Display)	Uzay + Zaman	Pencereler uzayda, odak geçişi zamanda çoğullanır

Soru 24 — write() Sistem Çağrısı — Kısmi Yazım

[EN] Can the write(fd, buffer, nbytes) call return any value in count other than nbytes? If so, why?

[TR] write() çağrısı nbytes'tan farklı bir değer döndürebilir mi?

Evet, write() nbytes'tan farklı bir değer döndürebilir:

Dönen Değer	Neden?
0 < count < nbytes	Disk doluysa, sinyal keserse (EINTR) veya ağ arabelleği doluysa kısmi yazım yapılır.
-1 (hata)	Disk hatası, izin yok, geçersiz fd — errno ayarlanır.

Soru 25 — lseek + read Örneği

[EN] A file contains: 2,7,1,8,2,8,1,8,2,8,4. `lseek(fd,3,SEEK_SET); read(fd,&buffer;,4)`. What does buffer contain?

[TR] lseek(fd,3,SEEK_SET) ve ardından read(fd,&buffer;,4) yapıldığında buffer ne içerir?

Dosya içeriği (indeks bazlı):

Konum: 0 1 2 3 4 5 6 7 8 9 10

Değer: 2 7 1 8 2 8 1 8 2 8 4

`lseek(fd, 3, SEEK_SET) → dosya konumu = 3. bayt`

`read(fd, &buffer;, 4) → konum 3'ten başlayarak 4 bayt oku`

Okunan baytlar: [3]=8, [4]=2, [5]=8, [6]=1

buffer = { 8, 2, 8, 1 }

Soru 26 — Dosya Okuma Süresi Hesabı

[EN] 10-MB file on track 50, disk arm at track 100. Arm moves 1 msec/cylinder, 5 msec rotational delay, 100 MB/s read rate. How long to retrieve the file?

[TR] 10 MB'lık dosyayı diskten okuma süresi nedir?

Seek süresi = $(100 - 50) \text{ silindir} \times 1 \text{ ms/silindir} = 50 \text{ ms}$

Dönme gecikmesi = 5 ms (verilmiş)

Transfer süresi = $10 \text{ MB} / 100 \text{ MB/s} = 100 \text{ ms}$

Toplam = $50 + 5 + 100 = 155 \text{ ms}$

Soru 27 — Blok ve Karakter Özel Dosyaları

[EN] What is the essential difference between a block special file and a character special file?

[TR] Blok özel dosya ile karakter özel dosya arasındaki temel fark nedir?

Özellik	Block Special File	Character Special File
Veri Erişimi	Sabit boyutlu bloklar halinde (512 B, 4 KB)	Karakter karakter (byte akışı)
Tampon (Buffer)	Kernel buffer cache kullanır	Doğrudan akış, tampon yok
Rastgele Erişim	Evet (lseek destekler)	Hayır (genellikle)

Örnek Aygıtlar	/dev/sda (disk)	/dev/tty (terminal), /dev/null
----------------	-----------------	--------------------------------

Soru 28 — Kütüphane Prosedürü ile Sistem Çağrısı Adlandırması

[EN] The library procedure is called read and the system call itself is called read. Is it essential that both have the same name?

[TR] Kütüphane prosedürü ve sistem çağrısının aynı adı taşıması şart mı?

Hayır, aynı ada sahip olmaları zorunlu değildir. POSIX standartlarına göre programcılar kütüphane arayüzünü kullanır; altta yatan sistem çağrısı farklı isimlendirilmiş olabilir. Örneğin Linux'ta bazı kütüphane fonksiyonları birden fazla sistem çağrısını sarmalar.

Daha önemli olan: Kütüphane prosedürünün adıdır. Kullanıcılar kütüphane API'siyle etkileşir. Sistem çağrısının adı uygulama programcısından gizlidir; yalnızca OS geliştiricileri için önemlidir.

Soru 29 — Client-Server Modeli — Tek Bilgisayar

[EN] The client-server model is popular in distributed systems. Can it also be used in a single-computer system?

[TR] Client-Server modeli tek bilgisayar sisteminde de kullanılabilir mi?

Evet. Microkernel tabanlı OS'lerde bu yaklaşım temeldir. Dosya sistemi, ağ yöneticisi gibi hizmetler ayrı süreçler (sunucu) olarak çalışır. Diğer süreçler (istemci) bunlara IPC (message passing) ile istek gönderir.

Örnek	İstemci	Sunucu
Dosya erişimi	Kullanıcı uygulaması	File server süreci
GUI (Wayland/X11)	Uygulama	Display compositor
Ses	Müzik çalar	PulseAudio daemon

Soru 30 — Sistem Çağrısı Farkındalığı

[EN] To a programmer, a system call looks like any other call to a library procedure. Is it important that a programmer know which library procedures result in system calls?

[TR] Programcının hangi kütüphane çağrılarının sistem çağrısına dönüştüğünü bilmesi önemli mi?

Genellikle önemlidir çünkü sistem çağrıları kullanıcı-kernel geçişi nedeniyle sıradan kütüphane çağrılarından çok daha yavaştır. Performans açısından kritik döngülerde sistem çağrılarından kaçınmak veya toplu hale getirmek (örn. küçük tek tek write() yerine büyük bir write()) önemlidir.



Örnek maliyet farkı: Kullanıcı-uzay fonksiyon çağırısı $\approx 1-5$ ns; sistem çağırısı (context switch dahil) $\approx 100-1000$ ns.

Soru 31 — UNIX → Win32 Dönüşüm Sonuçları

[EN] For UNIX system calls with no Win32 equivalent, what are the consequences for a programmer converting a UNIX program to run under Windows?

[TR] Win32 karşılığı olmayan UNIX sistem çağrıları için dönüşümün sonuçları nelerdir?

UNIX Çağırısı	Win32 Durumu	Sonuç
fork()	Yok (CreateProcess farklı)	Yeniden yazım gerekir; semantik farklı
exec()	Kısmi karşılık	Süreç oluşturma mantığı değiştirilmeli
Sinyaller (kill, signal)	Kısmi (SetEvent vb.)	Sinyal semantiği farklı; emüle edilmeli
chmod/chown	Yok (ACL tabanlı)	Güvenlik modeli farklı; yeniden tasarım
symlink/hardlink	Kısmi destek	Davranış tutarsızlıkları çözülmeli

Soru 32 — Taşınabilir İşletim Sistemi Tasarımı

[EN] Explain why it is infeasible to build a completely portable OS. Describe two high-level layers for a highly portable OS.

[TR] Tamamen taşınabilir OS neden imkânsızdır? Yüksek taşınabilirlik için iki katman nedir?

Donanıma özgü ayrıntılar (kayıt yapısı, interrupt mekanizması, MMU) doğrudan assembler/C kodu gerektirir; bunlar kesinlikle platforma özgüdür.

Katman	İçerik	Taşınabilirlik
1. HAL (Hardware Abstraction Layer)	CPU, bellek denetleyicisi, interrupt gibi donanım ayrıntılarını gizler.	Platforma özgü; her CPU mimarisi için yeniden yazılır
2. Üst Katman (Makine Bağımsız Kernel)	Zamanlayıcı, dosya sistemi, IPC — standart C ile yazılır.	HAL üzerine taşınabilir; çok az değişiklik gerekir

Soru 33 — Politika ve Mekanizma Ayrımı — Microkernel

[EN] Explain how separation of policy and mechanism aids in building microkernel-based operating systems.

[TR] Politika-mekanizma ayrımı microkernel tabanlı OS tasarımına nasıl katkı sağlar?

Kavram	Tanım	Örnek
--------	-------	-------

Mekanizma (Mechanism)	Ne yapılabileceğini tanımlayan düşük seviyeli araç	CPU zamanlama algoritması (round-robin çalıştırma)
Politika (Policy)	Ne yapılması gerektiğini belirleyen karar	Hangi süreç öncelikli çalışsın? (gerçek zamanlı mı?)

✓ Microkernel'de yalnızca mekanizmalar çekirdeğe gömülür; politikalar kullanıcı uzayındaki sunucu süreçlerde uygulanır. Bu sayede politika değiştirilmek istendiğinde kernel yeniden derlenmek zorunda kalınmaz → esneklik ve bakım kolaylığı.

Soru 34 — Sanal Makinelerin Dezavantajları

[EN] Virtual machines have become very popular. Nevertheless, they have some downsides. Name one.

[TR] Sanal makinelerin dezavantajlarından birini adlandırın.

Sanal makinelerin bazı dezavantajları şunlardır:

Dezavantaj	Açıklama
Performans Yüğü (Overhead)	Hypervisor katmanı ek işlem ve bellek gerektirir; özellikle yoğun I/O iş yüklerinde %5-30 yavaşlama gözlemlenebilir.
Lisans Karmaşıklığı	Her VM ayrı işletim sistemi lisansı gerektirebilir.
Güvenlik İzolasyonu	VM escape saldırıları ile izolasyon kırılabilir (nadiren ama mümkün).
Bellek Verimsizliği	Her VM kendi OS'u için bellek ayırır; çok sayıda VM büyük bellek baskısı oluşturur.

Soru 35 — Birim Dönüşümleri

[EN] Unit conversions: (a) microyear in seconds, (b) gigameter in meters, (c) bytes in 1-TB memory, (d) 6000 yottagrams in grams.

[TR] Birim dönüşüm hesaplamalarını yapın.

(a) 1 yıl = $365,25 \times 24 \times 3600 = 31.557.600$ s
1 mikroyıl = $31.557.600 \times 10^{-6} \approx 31,56$ saniye

(b) 1 gigameter = $10^9 \times 10^{-6} \text{ m} = 10^3 \text{ m} = 1 \text{ km}$

(c) 1 TB = 2^{40} bayt = $1.099.511.627.776$ bayt $\approx 1,1 \times 10^{12}$ bayt

(d) 1 yottagram = 10^{24} gram
6000 yottagram = $6 \times 10^3 \times 10^{24} = 6 \times 10^{27}$ gram

Soru 36-38 — Programlama Egzersizleri

[EN] Write a shell that actually works (with redirection, pipes, background jobs).

[TR] Gerçekten çalışan bir shell yazın (yönlendirme, pipe, arka plan desteği ile).

Aşağıda temel bir UNIX shell'inin C iskelet kodu verilmiştir:

```
#include
#include
#include
#include
#include
#include

void parse_and_exec(char *line) {
    int bg = 0;
    // Arka plan kontrolü
    if (line[strlen(line)-1] == '&') {
        bg = 1; line[strlen(line)-1] = '\0';
    }
    char *args[64]; int i = 0;
    char *tok = strtok(line, " \t\n");
    while (tok) { args[i++] = tok; tok = strtok(NULL, " \t\n"); }
    args[i] = NULL;
    if (!args[0]) return;
    pid_t pid = fork();
    if (pid == 0) { execvp(args[0], args); exit(1); }
    if (!bg) waitpid(pid, NULL, 0);
}

int main() {
    char line[1024];
    while (1) {
        printf("mysh> "); fflush(stdout);
        if (!fgets(line, sizeof(line), stdin)) break;
        parse_and_exec(line);
    }
}
```

[EN] Write a shell script that attempts to create an unlimited number of child processes.

[TR] Sınırsız sayıda alt süreç oluşturmaya çalışan shell betiği yazın.



UYARI: Bu betiği yalnızca kontrollü bir sanal makine ortamında deneyin. Gerçek sistem üzerinde çalıştırmak fork bomb etkisi yaratır.

```
#!/bin/bash
# Fork Bomb – Sonsuz alt süreç
# Çalıştırmadan önce: sync (disk bufferlarını temizle)
while true; do
  bash & # Her döngüde yeni bir bash süreci başlat
done

# Ya da tek satır versiyonu:
# :(){ :|:& };:
```

[EN] Examine the contents of a UNIX-like or Windows directory with the *od* program.

[TR] od gibi araçlarla dizin içeriğini inceleyin.

```
# Linux üzerinde dizin içeriğini ham (raw) görme
# Yöntem 1: od ile
$ sudo od -c /dev/sda | head -20

# Yöntem 2: debugfs (ext4 için)
$ sudo debugfs /dev/sda1
debugfs> ls -l /

# Yöntem 3: xxd ile
$ sudo xxd /dev/sda | head -30
```

Açık Uçlu Quiz Soruları ve Cevapları

i

Aşağıdaki sorular sınav hazırlığı için tasarlanmıştır. Önce cevaplamayı deneyin, ardından açıklama kısmını okuyun.

S1: İşletim sistemi olmaksızın bir bilgisayar kullanmak mümkün müdür?

Açıklayın.

Cevap: Evet, teorik olarak mümkündür — gömülü sistemlerde bazen 'bare metal' programlama yapılır. Ancak kullanıcı standart uygulamaları çalıştırmak, dosya sistemi kullanmak veya donanımı yönetmek istiyorsa OS zorunludur. OS olmadan programcı donanımın tüm ayrıntılarını elle yönetmek zorunda kalır.

S2: Bir süreç ile iş parçacığı (thread) arasındaki temel farkı açıklayın. Ne zaman thread, ne zaman process tercih edilir?

Cevap: Süreç: bağımsız adres alanına sahip izole yürütme birimi. Thread: aynı sürecin adres alanını paylaşan hafif yürütme birimi. Thread oluşturma daha hızlıdır ve iletişim kolaydır (paylaşılan bellek). Yüksek izolasyon gerekiyorsa (farklı güven seviyeleri) process tercih edilir.

S3: Sanal bellek (virtual memory) olmaksızın çoklu programlama mümkün müdür? Dezavantajları nelerdir?

Cevap: Evet, mümkündür (ilk çoklu programlama sistemleri sanal bellek kullanmıyordu). Dezavantajlar: bir süreç başka bir sürecin belleğine müdahale edebilir (güvenlik sorunu), toplam RAM programların toplam boyutunu sınırlar, süreçler bellekte sürekli kalmak zorundadır (swapping güç).

S4: Kernel modundan kullanıcı moduna geçiş ne zaman gerçekleşir? Tam akışı adım adım açıklayın.

Cevap: 1) Kullanıcı programı sistem çağrısı yapar (örn. read()). 2) CPU trap instruction (int 0x80, syscall) çalıştırır → kernel moduna geçer. 3) Kernel sistem çağrısı numarasını okur, ilgili handler'ı bulur. 4) Handler işlemi yapar. 5) Sonuç kullanıcı programına döndürülür, CPU kullanıcı moduna döner.

S5: Monolitik kernel ile microkernel arasındaki performans farkını açıklayın. Hangisi daha hızlıdır ve neden?

Cevap: Monolitik kernel genellikle daha hızlıdır: tüm OS servisleri aynı adres alanında çalışır, fonksiyon çağrısı maliyeti düşüktür. Microkernel'de servisler arası iletişim IPC (message passing) gerektirir; bu context switch ve mesaj kopyalama maliyeti ekler. Ancak microkernel daha güvenilir ve modüler olduğundan güvenlik açısından kritik sistemlerde tercih edilir.

S6: DMA denetleyicisi bir transfer tamamladığında işlemciye nasıl bildirim yapar? Bu mekanizmayı polling ile karşılaştırın.

Cevap: DMA interrupt (donanım kesmesi) gönderir. CPU, DMA tamamlandığında çalıştırdığı işi keser, interrupt handler'ı çalıştırır ve transferi tamamlar. Polling'de CPU sürekli 'transfer bitti mi?' diye kontrol eder → CPU israf olur. Interrupt yaklaşımı CPU'nun arada başka işlem yapmasına izin verir → verimlilik artar.

S7: Bir işletim sisteminde deadlock nasıl oluşur? Oluşmasını önlemeye yönelik dört koşuldan herhangi ikisini açıklayın.

Cevap: Deadlock: iki veya daha fazla sürecin birbirinin tuttuğu kaynağı bekleyerek sonsuz beklemeye düşmesi. Koşullar: (1) Mutual exclusion — kaynak aynı anda yalnızca bir süreç tarafından tutulabilir. (2) Hold & Wait — süreç kaynak tutarken başka kaynak bekleyebilir. (3) No preemption — kaynak süreç bırakmadan alınamaz. (4) Circular wait — döngüsel bekleme zinciri oluşur.

S8: Bellek koruma (memory protection) olmadan bir OS'de güvenli çok kullanıcı çalışması mümkün müdür? Neden?

Cevap: Hayır. Bellek koruma olmadan bir kullanıcının programı başka bir kullanıcının belleğini okuyabilir veya yazabilir. Bu hem gizlilik (şifre okuma) hem bütünlük (veri bozma) açısından ciddi güvenlik açığıdır. MMU ve kernel destekli bellek koruma çok kullanıcı sistemlerin temel gereksinimidir.
